

LAB-001

Safe Junos Configuration Workflow

SOLUTION GUIDE

Each task is followed immediately by its step-by-step configuration and verification workflow

Platform: Juniper vJunos-router 25.4R1.12

Environment: Containerlab

Credential: admin / admin@123

Task 0 - Deploy and baseline

Task requirements

- Deploy `lab001.clab.yml`.
- Wait until both Junos nodes are healthy.
- Connect to R1 through its Containerlab management address.
- Establish a separate console session to R1.
- Verify all four destinations using `scripts/verify.sh`.
- Record:
 - software version;
 - system uptime;
 - interface state;
 - route to each connected subnet;
 - current commit history;
 - logged-in users.

Evidence

- Deployment output
- Baseline verification output
- R1 operational command output

Step-by-step configuration and verification

Deploy from the directory containing `lab001.clab.yml`:

```
containerlab destroy -t lab001.clab.yml --cleanup
containerlab deploy -t lab001.clab.yml
docker logs -f r1
```

Wait for Junos startup to complete, then open management and console sessions:

```
ssh admin@r1
docker exec -it r1 telnet 127.0.0.1 5000
```

Run the baseline data-plane test from the host:

```
chmod +x scripts/verify.sh
./scripts/verify.sh
```

Collect the R1 baseline:

```
show version
show system uptime
show system users
show system commit
show interfaces terse
show route 192.168.10.0/24 exact
show route 10.0.12.0/30 exact
show route 10.255.0.1/32 exact
```

Expected client reachability:

Destination | Expected

--- | ---

192.168.10.1 | Reachable

10.0.12.2 | Reachable

10.255.0.1 | Reachable

10.255.0.2 | Reachable through R1 and R2 return route

Task 1 - Candidate versus active configuration

Task requirements

On R1, place the following intended information in the candidate configuration without committing it:

- Building: `JNCIE-LAB`
- Floor: `LAB-001`
- R1-to-R2 interface description: `SP-CORE-LINK`

Prove that:

- The candidate contains the change.
- The active configuration does not yet contain the change.
- Packet forwarding continues to use the active configuration.
- Discarding the candidate removes all pending changes.

Questions

- Which database is modified in configuration mode?
- Does editing a candidate immediately notify routing protocols?
- What is the safest way to discard all uncommitted changes?

Evidence

- Candidate configuration
- Candidate-to-active diff
- Active configuration from operational mode
- Verification before and after discarding the candidate

Step-by-step configuration and verification

```
configure
set system location building JNCIE-LAB
set system location floor LAB-001
set interfaces ge-0/0/1 description SP-CORE-LINK
show | compare
run show configuration system location
run show configuration interfaces ge-0/0/1
rollback 0
show | compare
```

`show` in configuration mode reads the candidate view. `run show configuration` reads the committed configuration through operational mode. `rollback 0` reloads the current active configuration into the candidate, thereby discarding uncommitted changes.

Task 2 - Validation failure and commit audit trail

Task requirements

Create a policy named `LAB001-TEST` that references a prefix list that does not exist.

- Validate without activating the configuration.
- Capture the complete error.
- Create the missing prefix list containing `192.0.2.0/24`.
- Validate again.
- Review the candidate diff.
- Commit using an audit comment containing the lab ID and change purpose.
- Prove the commit is present in history.

Do not apply the test policy to a routing protocol.

Evidence

- Failed validation
- Corrected validation
- Diff
- Commit history and comment

Step-by-step configuration and verification

```
set policy-options policy-statement LAB001-TEST term 10 from prefix-list LAB001-PREFIXES
set policy-options policy-statement LAB001-TEST term 10 then accept
commit check
set policy-options prefix-list LAB001-PREFIXES 192.0.2.0/24
commit check
show | compare
commit comment "LAB-001: create isolated policy validation example"
run show system commit
```

The first validation should fail because the referenced prefix list is undefined. A successful `commit check` proves configuration validation, not service correctness. Operational post-checks remain mandatory.

Task 3 - Automatic rollback

Task requirements

Add rack location `RACK-01`.

- Activate it temporarily with a three-minute confirmation window.
- Prove the value is active.
- Identify the pending rollback deadline.
- Do not confirm it.
- Prove the value disappears after timeout.
- Repeat the change.
- This time, verify it and make it permanent before timeout.

Questions

- Is a confirmed commit active or merely staged?
- What event makes the change permanent?
- What happens when the session closes before confirmation?

Step-by-step configuration and verification

First attempt, intentionally allow rollback:

```
set system location rack RACK-01
show | compare
commit check
commit confirmed 3
run show system commit
run show configuration system location
```

After the timeout:

```
run show configuration system location
run show system commit
```

Second attempt, confirm after verification:

```
set system location rack RACK-01
commit confirmed 3 comment "LAB-001: temporary rack change"
run show configuration system location
commit comment "LAB-001: confirm verified rack change"
```

A confirmed commit is active during the confirmation interval. An ordinary `commit` before the deadline confirms it permanently. If not confirmed, Junos automatically restores the preceding committed configuration.

Task 4 - Management-loss simulation

Task requirements

Perform this task only with a working console session.

Client1 reaches R1 through `192.168.10.1`. Change R1's client-facing address to `192.168.99.1/24` using a two-minute automatic recovery window.

Requirements:

- Perform and record pre-checks.
- Review the exact diff.
- Validate the candidate.
- Activate it temporarily.
- Prove client1 loses reachability.
- Do not correct the address through console.
- Observe the pending commit through console.
- Prove the original address and reachability return automatically.

Incident note

Record:

- Symptom
- Impact
- Initial hypothesis
- Evidence
- Recovery mechanism
- Post-recovery verification

Step-by-step configuration and verification

Pre-check:

```
show system users
show interfaces ge-0/0/0 terse
show route 192.168.10.0/24 exact
show system commit
```

Change:

```
configure
delete interfaces ge-0/0/0 unit 0 family inet address 192.168.10.1/24
set interfaces ge-0/0/0 unit 0 family inet address 192.168.99.1/24
show | compare
commit check
commit confirmed 2 comment "LAB-001: management-loss recovery test"
```

From the console, observe:

```
show system commit
```

```
show interfaces ge-0/0/0 terse
```

After timeout, expected state:

```
show interfaces ge-0/0/0 terse
show configuration interfaces ge-0/0/0
```

`192.168.10.1/24` should return, and `scripts/verify.sh` should pass again. If you use an ordinary commit, no automatic recovery timer exists and console/manual rollback is required.

Task 5 - Manual rollback

Task requirements

Create and commit three distinct rack values, with a unique commit comment for each.

Then:

- Display commit history.
- Load the immediately preceding committed configuration into the candidate.
- Review the reverse diff before activation.
- Commit the rollback with an audit comment.
- Load an older rollback revision and inspect it without activating it.
- Return the candidate to the current active configuration.
- Prove there is no remaining candidate diff.

Questions

- Does loading a rollback immediately change forwarding?
- What does rollback zero represent?
- Why must a rollback be reviewed before commit?

Step-by-step configuration and verification

```
configure
set system location rack RACK-01
commit comment "LAB-001: rack revision 1"
set system location rack RACK-02
commit comment "LAB-001: rack revision 2"
set system location rack RACK-03
commit comment "LAB-001: rack revision 3"
run show system commit
rollback 1
show | compare
commit check
commit comment "LAB-001: restore preceding rack revision"
rollback 2
show | compare
rollback 0
show | compare
```

Loading a rollback modifies only the candidate. The loaded revision becomes active only after commit. Rollback zero represents the currently active committed configuration.

Task 6 - Rescue configuration

Task requirements

- Confirm the router is in a known-good state.
- Save the known-good committed configuration as the rescue configuration.
- Prove the rescue configuration exists.
- Commit an intentionally incorrect rack value.
- Load the rescue configuration into the candidate.
- Inspect the diff.
- Validate and commit the restoration with an audit comment.
- Run the complete verification script.

Questions

- How is rescue different from rollback one?
- When should rescue be refreshed?
- What is the risk of restoring a stale rescue configuration?

Step-by-step configuration and verification

Save the current known-good configuration from operational mode:

```
request system configuration rescue save
show system configuration rescue
```

Create and restore an intentional bad value:

```
configure
set system location rack WRONG-RACK
commit comment "LAB-001: intentional rescue test"
rollback rescue
show | compare
commit check
commit comment "LAB-001: restore known-good rescue configuration"
```

Rollback history is chronological and changes after every commit. Rescue is an explicitly saved known-good checkpoint. A stale rescue configuration can remove legitimate changes made after it was saved, so always inspect the diff before committing a rescue restoration.

Task 7 - Concurrent configuration sessions

Task requirements

Open two SSH sessions to R1.

Shared candidate

- Enter standard configuration mode from both sessions.
- Make a location change in session A without committing.
- Determine whether session B can see it.
- Discard all changes.

Exclusive candidate

- Acquire an exclusive configuration lock in session A.
- Attempt to enter configuration mode from session B.
- Record the result and identify the lock owner.

Private candidates

- Enter private configuration mode from both sessions.
- Modify different location leaves.
- Compare visibility between sessions.
- Commit session A.
- Review and commit session B.
- Repeat using conflicting edits to the same leaf and document the result.

Step-by-step configuration and verification

Shared

Session A:

```
configure
set system location building SHARED-A
show | compare
```

Session B:

```
configure
show | compare
```

Both sessions operate on the shared candidate, so session B should see session A's uncommitted change. Clean up from either session:

```
rollback 0
exit
```

Exclusive

```
configure exclusive
```

Keep session A in exclusive mode. In session B, run `configure` and record the lock error. In session A, identify users and release the lock:

```
run show system users  
exit
```

Private

```
configure private
```

Session A:

```
configure private  
set system location building PRIVATE-A  
show | compare  
commit comment "LAB-001: private session A"
```

Session B:

```
configure private  
set system location floor PRIVATE-B  
show | compare  
commit check  
commit comment "LAB-001: private session B"
```

Repeat with both sessions changing `system location rack` to different values. Commit session A first; then run `show | compare` and `commit check` in session B and record the conflict or merge result before cleanup. Each private candidate is derived from committed configuration, and Junos rejects changes it cannot merge safely.

Final assessment - Blind workflow

Task requirements

Without using the solution guide, perform this change request:

Change the R1-to-R2 interface description to `PROD-CORE-R1-R2`. Maintain service availability, include an audit trail, and demonstrate rollback readiness.

Submit:

- Change plan
- Pre-check output
- Candidate diff
- Validation result
- Activation method and justification
- Post-check output
- Rollback plan
- Final commit history

Scoring

Dimension | Weight

--- | ---:

Candidate/active understanding | 10%

Safe commit workflow | 20%

Confirmed commit recovery | 20%

Manual rollback | 15%

Rescue configuration | 10%

Concurrent sessions | 10%

Operational verification | 10%

Documentation | 5%

Pass requires at least 80% and no critical recovery or verification failure.

Step-by-step configuration and verification

```
show interfaces ge-0/0/1 extensive
show route 10.0.12.0/30 exact
ping 10.0.12.2 rapid count 5
configure exclusive
set interfaces ge-0/0/1 description PROD-CORE-R1-R2
show | compare
commit check
commit confirmed 3 comment "LAB-001: change R1-R2 link description"
run show interfaces ge-0/0/1 descriptions
run ping 10.0.12.2 rapid count 5
commit comment "LAB-001: confirm verified R1-R2 description"
run show system commit
```

The interface-description change is low risk, but using the complete workflow builds the habit required for higher-risk production changes.

Mastery check

You have mastered LAB-001 when you can explain and demonstrate all of the following without the guide:

- Candidate versus active configuration
- `commit check` versus operational verification
- Confirmed commit activation and confirmation
- Automatic rollback after management loss
- Rollback zero and previous revisions
- Rescue configuration lifecycle
- Shared, exclusive and private edit sessions
- Audit comments, evidence collection and post-checks